

Kiste Phase 2 Spec

Remote Repo Scanner + Kiste CI Testing

Phase 2 adds two major capabilities: GitHub Actions testing for the Kiste codebase, and provider-aware scanning for GitHub and Hugging Face repositories.

One-sentence spec: **Kiste Phase 2 adds CI testing for the Kiste codebase and upgrades the scanner from local-only scanning to provider-aware scanning for GitHub and Hugging Face repositories.**

1. Phase 2 Goal

Kiste should no longer scan only local folders. It should understand repo sources from local disk, GitHub, and Hugging Face Hub, then run the same scanner pipeline on important files from each source.

```
kiste scan .
kiste scan github owner/repo
kiste scan github owner/repo --ref main --token github-dev
kiste scan hf owner/model-name
kiste scan hf owner/dataset-name --type dataset
kiste scan hf owner/space-name --type space
```

Kiste itself should also gain CI protection so every push or pull request runs install, lint, type check, tests, and CLI smoke tests.

2. Scope

Included

- GitHub Actions workflow for the Kiste repo.
- Provider abstraction for local, GitHub, and Hugging Face repositories.
- GitHub repo scanning through selected important files.
- Hugging Face model, dataset, and Space scanning.
- Hugging Face token metadata support.
- Mocked tests for provider APIs.
- Fixture-based scanner tests for common repo types.

Excluded

- Real Kubernetes deployment.
- Terraform apply or cloud provisioning.
- Executing code from scanned repos.
- Full model security analysis.
- Full cost engine and GPU scheduler.

3. GitHub Actions for Kiste Repo

Add a CI workflow at:

```
.github/
  workflows/
    ci.yml
```

Recommended CI workflow:

```
name: Kiste CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]

    steps:
      - name: Checkout repo
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Install Kiste
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Lint
        run: ruff check .

      - name: Type check
        run: mypy kiste_cli

      - name: Run tests
        run: pytest -q

      - name: CLI smoke test
        run: |
          kiste --help
          kiste scan tests/fixtures/fastapi_docker_repo
```

Dev dependencies

```
[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "ruff>=0.6.0",
    "mypy>=1.10.0",
    "responses>=0.25.0"
]
```

4. New Command Tree

Phase 1 command tree:

```
kiste
  init
  scan
  check
  repo
  token
```

Phase 2 command tree:

```
kiste
  scan
    local
    github
    hf
  provider
    github
    inspect
```

```
hf
  inspect
token
  add
  list
inspect
check
```

For better user experience, support both direct and explicit scan styles:

```
kiste scan .
kiste scan local .
kiste scan github owner/repo
kiste scan hf owner/repo
```

5. GitHub Repo Scan

Command examples:

```
kiste scan github owner/repo
kiste scan github owner/repo --ref main
kiste scan github owner/repo --token github-dev
```

Expected output:

GitHub repo scan:

```
[OK] Repo found: owner/repo
[OK] Dockerfile found
[OK] docker-compose.yml found
[OK] requirements.txt found
[OK] FastAPI detected
[OK] Port 8000 detected
[WARN] Secrets detected: DATABASE_URL, SECRET_KEY
[OK] K8s readiness: Deployment, Service, ConfigMap, Secret template
[FAIL] Ingress host missing
```

Important files to fetch first

- Dockerfile
- docker-compose.yml / compose.yml
- requirements.txt / pyproject.toml
- package.json
- go.mod
- README.md
- .env.example
- .github/workflows/*

Do not clone the whole repo first unless needed. Kiste should prefer reading selected important files through the provider interface.

GitHub scan spec addition

```
{
  "source": {
    "provider": "github",
    "repo": "owner/repo",
    "ref": "main"
  }
}
```

6. Hugging Face Repo Scan

Command examples:

```
kiste scan hf owner/model-name
```

```
kiste scan hf owner/dataset-name --type dataset
kiste scan hf owner/space-name --type space
kiste scan hf owner/private-model --token hf-dev
```

HF scan should detect

- model repo, dataset repo, or Space repo
- README.md / model card
- config.json and model_index.json
- *.safetensors and *.bin files
- requirements.txt and app.py
- Dockerfile if present
- Space runtime hints
- GPU hints
- large files / LFS files

Example model output

Hugging Face repo scan:

```
[OK] Repo found: owner/model-name
[OK] Repo type: model
[OK] README.md found
[OK] config.json found
[OK] safetensors model file found
[WARN] Large model artifact detected
[WARN] GPU likely required
[FAIL] No Dockerfile found
```

Example Space output

Hugging Face Space scan:

```
[OK] Repo found: owner/space-name
[OK] app.py found
[OK] requirements.txt found
[OK] Gradio detected
[OK] Runtime: Python Space
[WARN] GPU requested in README metadata
```

Safety rule: Kiste scans HF files as data. It must not execute repo code during scan.

7. Provider Abstraction

Add an internal provider interface so the scanner does not care where files come from.

```
RepoProvider
- LocalProvider
- GitHubProvider
- HuggingFaceProvider
```

Interface

```
class RepoProvider:
    def list_files(self) -> list[str]:
        ...

    def read_file(self, path: str) -> str | bytes | None:
        ...

    def repo_metadata(self) -> dict:
        ...
```

Unified pipeline

```
provider
-> file inventory
-> important file reader
-> spec builder
-> readiness checker
-> report
```

8. Token Updates

Phase 2 adds Hugging Face token metadata support beside GitHub token metadata.

```
kiste token add hf --name hf-dev --scope repo:read --expires 2026-08-01
```

```
kiste token list
```

Example output:

NAME	PROVIDER	SCOPES	EXPIRES	STATUS
github-dev	github	repo:read,packages:write	2026-08-01	active
hf-dev	hf	repo:read	2026-08-01	active

Raw tokens should still not be stored in plaintext. Use environment variables:

```
export KISTE_TOKEN_GITHUB_DEV="github-token"
export KISTE_TOKEN_HF_DEV="hf-token"
```

9. Phase 2 Test Plan

Test folders

```
tests/
  fixtures/
    fastapi_docker_repo/
      Dockerfile
      requirements.txt
      main.py
      docker-compose.yml

    nextjs_docker_repo/
      Dockerfile
      package.json

    hf_space_repo/
      app.py
      requirements.txt
      README.md

  test_local_scan.py
  test_github_provider.py
  test_hf_provider.py
  test_token_metadata.py
  test_cli.py
```

Test cases

- Local scan detects FastAPI.
- Local scan detects Dockerfile port.
- Local scan detects Postgres and Redis from Compose.
- GitHub provider lists files using mocked API.
- GitHub provider reads Dockerfile using mocked API.
- HF provider lists model repo files using mocked API.
- HF provider detects safetensors/bin files.
- CLI returns success for kiste scan.
- CLI returns warning for missing Dockerfile.

10. Phase 2 Acceptance Criteria

Phase 2 is complete when these commands work:

```
kiste scan .
kiste scan github owner/repo
kiste scan github owner/repo --ref main --token github-dev
kiste scan hf owner/model-name
kiste scan hf owner/dataset-name --type dataset
kiste scan hf owner/space-name --type space
kiste token add hf --name hf-dev --scope repo:read --expires 2026-08-01
```

And GitHub Actions runs automatically:

- [OK] install
- [OK] lint
- [OK] type check
- [OK] tests
- [OK] CLI smoke test
- [OK] fixture scan test

11. Suggested Phase 2 File Changes

```
kiste_cli/
  providers/
    __init__.py
    base.py
    local.py
    github.py
    huggingface.py
  scanner.py
  tokens.py
  main.py

.github/
  workflows/
    ci.yml

tests/
  fixtures/
    test_local_scan.py
    test_github_provider.py
    test_hf_provider.py
    test_cli.py
```

12. Phase 2 Design Principle

Kiste should treat remote repositories as file systems. GitHub, Hugging Face, and local folders should all expose a common file inventory and file-reading interface. This keeps the scanner reusable and makes future providers easier to add.

Future providers can include GitLab, Bitbucket, S3, Google Drive, private artifact stores, and internal enterprise Git servers.